REACT UI PROJECT FINAL EVALUATION

20 MARKS
PROJECT-SPECIFIC FIELD GUIDE

TradingView Clone | React + Vite frontend, Express + MongoDB backend

A concise explanation of how the checked-out project is structured, how its real screens behave, and how to demonstrate your individual understanding against the supplied 20-mark rubric. Every path and line range in this guide refers to the source present in the project workspace when this PDF was prepared.

> **Use this as a map, not a script.** Be ready to open the cited source, explain why the state or effect exists, and describe one failure case and recovery for the feature you demonstrate.

Prepared 28 October 2026 | Includes verified routes, form behavior, selected code excerpts, demo outline, viva practice, and evidence/limitations.

Contents

1. What the rubric rewards 3

2. Architecture: how a request becomes a screen 4

3. Criterion 1 - Component design and rendering 5

4. Criterion 2 - UI styling and responsiveness 6

5. Criterion 3 - Hooks and state management 7

6. Criterion 4 - Routing and forms 8

7. Walk-through: sign-up and chart interactions 10

8. Code reference index 11

9. A 3-5 minute demonstration plan 12

10. Viva practice - project-specific questions 13

11. Debugging, verification, and honest limits 15

12. Final preparation checklist 16

> Evidence rule: paths and line references are taken from source inspection. Build and backend test results are recorded near the end; those results do not prove external providers or live MongoDB behavior.

1. What the rubric rewards

The supplied rubric has five equally weighted criteria. Each criterion is scored from 1 to 4, for 20 marks total. The strongest demonstration connects a visible behavior to the component, state, event, and styling that implement it.

Criterion	Excellent (4) evidence to show
1. Component design and rendering	Reusable components; props used correctly; conditional rendering works.
2. UI styling and responsiveness	Consistent styling; appropriate dynamic styles; responsive layout.
3. Hooks and state management	Correct hooks; clear state ownership; event handling updates UI as expected.
4. Routing and forms	Routes navigate correctly; controlled form inputs, validation, submission work.
5. Individual understanding and demonstration	Explain implementation confidently; debug independently; identify your contribution.

Before the evaluation, rehearse one feature per criterion and know its source location. Do not claim a feature is complete merely because a route exists: show its success, empty/loading, and error behavior when possible.

Project evidence	Preparation status
Reusable composition	Market pages reuse shared components; ChartWorkspace composes tab, toolbar, chart, and sidebar components.
State and effects	React useState/useEffect/useMemo/useCallback/useRef appear in active pages and components.
Routing/forms	React Router route table and multiple real submit handlers; most business forms use controlled values.
Styling	Large shared stylesheet contains responsive breakpoints, focus states, and reduced-motion handling.
Tests/build	Current backend suite: 58 passing. Frontend production build: run during preparation; see recorded result in section 11.

2. Architecture: how a request becomes a screen

The browser entry creates the React tree. App supplies routing and authentication context, then route components call the API service. The Vite development server proxies /api to Express. Express routes validate/authenticate requests and use services/models or external providers.

Layer	Project location	Responsibility
Browser entry	frontend/src/main.jsx	Mounts App inside React StrictMode and imports shared styles.
Route and auth shell	frontend/src/App.jsx	BrowserRouter, AuthProvider, lazy page imports, route mapping, protected page guard.
Pages	frontend/src/pages/	Route-level UI and page state: markets, chart, auth, paper trading, ideas, India and FX.
Reusable UI	frontend/src/components/	Shared navigation/status/market strip, community panels, market overview hero/chart.
Client API	frontend/src/services/api.js	Fetch wrapper, API root, token header, parsing and error normalization.
Backend	backend/server.js; backend/routes/; backend/services/; backend/models/	Express endpoints, domain logic, provider adapters, schemas and persistence.

Flow	Sequence
Initial page	main.jsx -> App.jsx -> route match -> page component -> shared components
Data fetch	page/effect -> services/api.js -> Vite /api proxy -> Express route -> service/provider or Mongo model -> JSON -> state update -> render
Protected action	ProtectedRoute/AuthContext -> token in api request -> authenticated backend route -> result/error shown in component

Small source tree

```
frontend/src/
  main.jsx
  App.jsx
  context/AuthContext.jsx
  services/api.js
  components/Shared.jsx
  components/Community.jsx
  components/MarketOverview.jsx
  pages/Markets.jsx
  pages/ChartWorkspace.jsx
  pages/AuthPages.jsx
  pages/PaperTrading.jsx
  pages/IndiaMarket.jsx / IndiaChart.jsx
  styles.css
backend/
  server.js | routes/ | services/ | models/ | middleware/
```

> Chart route detail: /chart redirects to /chart/BTCUSDT. The active /chart/:symbol route renders ChartWorkspace.jsx. There is a separate legacy Chart.jsx file; it is not the component mounted by the current route.

3. Criterion 1 - Component design and rendering

The project separates route-level work from reusable display and behavior. App.jsx wires routes, pages own page state, and components receive data and callbacks through props. The market overview uses the same hero and chart component with a market prop, while shared empty/loading/error components give market and community screens consistent states.

Example: a component receives data and configuration

```
export function MarketOverviewChart({ data = [], color, precision, loading, label, timezone }) {
  const hostRef = useRef(null);
  // Create chart in an effect; resize and remove it on cleanup.
  ...
  if (loading) return <LoadingState label={label} />;
  if (!data.length) return <EmptyState label={label} />;
  return <div ref={hostRef} className="market-overview-chart" />;
}
```

Source: frontend/src/components/MarketOverview.jsx:14-56. The excerpt is abbreviated at the chart setup; use the source file for the full effect. The defaults prevent missing data from crashing the component; the loading and empty branches make the render outcome explicit.

Example: page composition and callback props

```
const content = useMemo(() => {
  if (activeTab === 'watchlist') return <WatchlistTab selectedSymbol={symbol} onSelectSymbol={selectSymbol} />;
  if (activeTab === 'trade') return <TradeTab chartSymbol={symbol} portfolio={portfolio} ... />;
  if (activeTab === 'alerts') return <AlertsTab symbol={symbol} />;
  return <ProfileTab symbol={symbol} />;
}, [activeTab, symbol, portfolio, ...]);
```

Source: frontend/src/pages/ChartWorkspace.jsx:855-862. A parent selects the active child and passes the symbol, data, and callbacks it needs. A useful oral explanation: “The tab state belongs to the workspace because it decides which panel is visible; the child receives only the props needed to render or act.”

Render details worth pointing out

Pattern	Actual example	Why it matters
Conditional render	Loading / error / empty states in Shared.jsx and market components.	Prevents blank panels and provides an action or status.
List keys	Market rows keyed by symbol; article cards use stable article identity.	Lets React reconcile list updates predictably.
Reusable props	MarketOverviewHero({ market }); MarketOverviewChart({ data, color, ... }).	Same component can serve crypto/forex variants.
Children composition	AuthSplitLayout receives children and footer in AuthPages.jsx:6-20.	Layout stays separate from sign-in/sign-up behavior.

> For a code explanation, name the parent, props, state change, and render branch. Avoid saying “React updates it automatically” without identifying the event or state value that changed.

4. Criterion 2 - UI styling and responsiveness

The project styling is primarily in frontend/src/styles.css. It combines a dark trading workspace with light market overview panels. Responsive CSS reorganizes multi-column areas at several viewport widths, and the home hero includes motion styles with a reduced-motion preference.

Evidence in CSS	What to explain
Market cards/chart: styles.css:90-132	Grid/card/chart styling for market overview.
Market breakpoints: :145-164	Layouts adjust around 920, 680, and 450 px.
Focus visibility: around :231	Keyboard focus gets a visible treatment.
Home motion: around :456-468	Animation is reduced when prefers-reduced-motion is enabled.
Chart workspace: :510-729	Workspace-specific layout and breakpoints around 1120, 820, 560 px, plus later small-screen rules.
Auth layout: :733-776	Two-column layout transitions for tablet and phone widths.

How to demonstrate the layout

Open the crypto or forex overview and identify the reusable summary/chart areas. Narrow the browser and point to the relevant @media rules in CSS. Use keyboard Tab to show focus styling. If the home hero animation is present, explain its reduced-motion override.

> Verification limit: CSS breakpoints were inspected in source. The guide does not claim a live screenshot or device-by-device responsive QA. Be ready to test the actual page at a narrow viewport during the demo.

Styling improvement discussion

If asked what you would improve, identify a concrete small-screen layout or a contrast/focus detail, then state how you would validate it. The current app has mixed dark workspace and light market-card treatments; no user-facing theme selector was found in the inspected source.

5. Criterion 3 - Hooks and state management

State is local to the page or component that owns the interaction. Context carries authentication across the route tree. Effects load data or synchronize with browser/chart resources. Callbacks and memoization appear where child props or derived panels benefit from stable references.

Hook / state	Actual use	Explanation
useState	Markets.jsx:10-14; AuthPages.jsx:27,44; ChartWorkspace.jsx:153-160,785-800	Stores user-visible selections, request status, inputs and data.
useEffect	Markets.jsx:15-38; AuthContext.jsx mount flow; ChartSurface.jsx:177-222	Runs data/resource synchronization; dependencies define when it reruns.
useRef	ChartWorkspace.jsx:143-152; portfolioLock at :800	Stores chart objects or a lock without forcing a render.
useCallback	ChartWorkspace.jsx:162-175 and :811-850	Stabilizes event/data functions where needed.
useMemo	ChartWorkspace.jsx:855-862	Derives the active sidebar panel from current tab and inputs.
Context/custom hook	AuthContext.jsx: useAuth	Shares auth state and exposes login/register/logout through a focused API.

Concrete effect lifecycle: selected market history

```
useEffect(() => {
  const controller = new AbortController();
  api.marketHistory(selectedSymbol, controller.signal)
    .then((response) => setHistory(response.data || []))
    .catch((requestError) => {
      if (!controller.signal.aborted) setError(requestError.message);
    });
  return () => controller.abort();
}, [selectedSymbol]);
```

Source: frontend/src/pages/Markets.jsx:28-38. When selectedSymbol changes or the component unmounts, cleanup aborts the previous request. This prevents a stale result from overwriting the selected market state.

Concrete chart lifecycle

ChartSurface creates the Lightweight Charts instance in an effect, subscribes to resize/crosshair/range events, then disconnects the observer, unsubscribes listeners, closes the socket and removes the chart in cleanup (ChartWorkspace.jsx:177-222). Explain why cleanup matters: the chart and browser listeners are external resources, not just React values.

State update and event handling

toggleIndicator uses a functional state update at ChartWorkspace.jsx:844, so the new indicator object is derived from the latest state. selectSymbol normalizes and validates the input before navigation at :845-850. A controlled form input writes to state onChange; the render then reflects that state.

> A common viva follow-up: “Why not put everything in one global state?” Answer using this code: auth is cross-route and belongs in context; chart tab, selected interval, and input values are local to the workspace because unrelated pages do not need them.

6. Criterion 4 - Routing and forms

App.jsx defines the client route table under BrowserRouter and AuthProvider. Page modules are lazy-loaded; the root app uses a route frame and error/not-found states. ProtectedRoute waits for auth loading, then either renders the requested page or directs an unauthenticated visitor to sign in.

Route	Rendered page / behavior	Access
/	Home	Public
/markets/crypto	Markets (crypto overview)	Public
/markets/forex	Markets (forex overview)	Public
/markets/india	India market	Public
/markets/india/:exchange	India market, exchange-filtered	Public
/chart	Redirect to /chart/BTCUSDT	Public
/chart/:symbol	ChartWorkspace (active chart route)	Public shell; features may require login
/forex/:symbol	ForexChart	Public
/india/:exchange/:symbol	IndiaChart	Public
/india/:symbol	IndiaChart fallback route	Public
/ideas/:id	IdeaDetail	Public route; actions can require auth
/signin and /signup	AuthPages	Public
/features and /pricing	Feature/pricing pages	Public
/paper-trading	PaperTrading	Protected
*	NotFound	Public

Source: frontend/src/App.jsx:29-57. The exact nested elements and redirect are in source; route names above are summarized for quick revision.

Real form inventory

Form	Input behavior / validation	Submit and feedback
Sign in (AuthPages.jsx:30-36)	Controlled email/password; required browser validation.	login() -> user context; navigate to destination; busy/error state.
Sign up (AuthPages.jsx:46-54)	Controlled name (3-30 chars), email (type=email, max 254), password (8-72 chars); required.	register() -> context; navigate on success; busy/error state.
Chart symbol (ChartWork space.jsx:870-872)	Uncontrolled input with defaultValue/key and datalist; read from currentTarget on submit. selectSymbol validates uppercase USDT pair (:845-850).	Prevents default then navigates to /chart/:symbol.
Watchlist add (ChartWork space.jsx:582-589)	Query normalized uppercase, slash removed, regex checks USDT pair; sign-in required for saving.	API add, clear input and reload list; inline error on failure.
Crypto alert (ChartWorkspace.jsx:750; submit :738-746)	Controlled condition/target; target required, positive min; pair is fixed.	Saves alert, status/error message, refresh list.
Paper trade (PaperTrading.jsx:164+; form :276-289)	Controlled order fields; mode and side drive preview/availability.	Validates quote/amount/funds/holdings; submits order and refreshes portfolio.
India price alert (IndiaChart.jsx:113-132)	Controlled condition and target; required number, min 0.01, step 0.01.	Saves through India alert API; reports success/error.
Community comment (IdeaDetail.jsx:19-22)	Comment and edit actions use auth-aware state and length limit.	Post/update/delete calls; list keyed by comment id.
Forex comparison (ForexChart.jsx:143-162)	Controlled symbol text/datalist; button disabled if blank.	Loads comparison and reports provider/input errors.

> Controlled means React state supplies the input value and onChange updates it. The chart symbol form is an intentional contrast: it reads the submitted DOM value and resets via key={symbol}.

7. Walk-through: sign-up and chart interactions

For a quick source explanation, trace one form from the input to the API and back to visible UI. These excerpts show actual patterns; omitted UI markup is indicated by comments in the explanation, not replaced with claimed source.

Sign-up input constraints

```
<input value={name} onChange={(event) => setName(event.target.value)}
  required minLength={3} maxLength={30} />
<input type="email" value={email}
  onChange={(event) => setEmail(event.target.value)}
  required maxLength={254} />
<input type="password" value={password}
  onChange={(event) => setPassword(event.target.value)}
  required minLength={8} maxLength={72} />
```

Source: frontend/src/pages/AuthPages.jsx:48-53. The submit handler at :46 prevents default, calls register, navigates on success, and uses error/busy state. The backend also validates credentials; browser constraints improve the input experience but are not a security boundary.

API request behavior

```
const API_ROOT = import.meta.env.VITE_API_URL || '/api';
async function request(path, options = {}) {
  const token = localStorage.getItem('tv_react_token');
  const response = await fetch(`${API_ROOT}${path}`, {
    ...options, headers: { ...(options.headers || {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}) }
  });
  const payload = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(payload.message || 'Request failed');
  return payload;
}
```

Source: frontend/src/services/api.js:1-21. Mention that the service centralizes API root, token attachment, JSON parsing, and error conversion. Keep secrets out of the frontend; provider credentials belong in the backend environment.

Chart resource and indicator interaction

In ChartWorkspace, ChartSurface owns the third-party chart instance and keeps it in refs. The parent owns the chart type, interval, indicator toggles, active drawing tool, symbol, and per-symbol drawing data. A child callback lets drawing interactions update parent state. Drawing state is synchronized to localStorage (:792-794, :810). Explain the trade-off: it survives reloads in this browser, but it is not server-synced.

Auth protection flow

AuthProvider initializes the saved user/token state and exposes login/register/logout. ProtectedRoute first displays loading while authentication is being checked; it then returns the protected child for a user or a redirect to sign-in. Paper trading is protected at the route table. This is a UI gate; the backend must still enforce authentication on protected endpoints.

8. Code reference index

Use these exact source locations for exam follow-ups. Line numbers are from the checked project files at guide creation; if the project changes, search for the named component/function before relying on the number.

File	Lines	What is there
frontend/src/main.jsx	1-6	React root, StrictMode, App, global styles.
frontend/src/App.jsx	29-57	Auth/router shell, lazy route mapping, protected paper-trading route, not-found page.
frontend/src/context/AuthContext.jsx	whole file; useAuth near end	Shared user state, login/register/logout, auth check on mount.
frontend/src/services/api.js	1-21; methods from 29	Fetch wrapper, API root, bearer token, normalized errors, endpoint methods.
frontend/src/components/Shared.jsx	17-58; 60+; 142+	Sparkline, headings/status components, MarketStrip polling, Navbar, ProtectedRoute.
frontend/src/components/MarketOverview.jsx	6-56	Overview hero and chart lifecycle, resize observer, loading/empty branches.
frontend/src/components/Community.jsx	11-65; 77-123	Idea filter/card/feedback, abortable idea/news loads and category rendering.
frontend/src/pages/Markets.jsx	10-58	Quotes refresh, selected-symbol history, filters, summary/chart/list.
frontend/src/pages/AuthPages.jsx	6-20; 23-56	Shared auth layout; controlled sign-in and sign-up forms.
frontend/src/pages/Home.jsx	11-38; 95+	Market sparkline/summary state and polling; home hero composition.
frontend/src/pages/ChartWorkspace.jsx	142-222	Chart refs, creation, subscriptions and effect cleanup.
frontend/src/pages/ChartWorkspace.jsx	517-620	Watchlist state/load, India quotes, symbol validation and list actions.
frontend/src/pages/ChartWorkspace.jsx	722-752	Alerts state, submit path and form controls.
frontend/src/pages/ChartWorkspace.jsx	779-882	Workspace state, URL tab behavior, drawings, symbol validation and composition.
frontend/src/pages/PaperTrading.jsx	55-69; 94; 102; 164+; 276-289	Paper-trading state/effects, order action, user-facing form.
frontend/src/pages/IdeaDetail.jsx	11-22	Idea/comment load and mutations, comment conditional states.
frontend/src/pages/ForexChart.jsx	18-32; 41+; 143-162	FX quote/history state, effect lifecycle, comparison interaction.
frontend/src/pages/IndiaMarket.jsx	9-10; 17-29; 50+; 111-112	INR/time formatting, India market filters and rows.
frontend/src/pages/IndiaChart.jsx	19-55; 113-132	India chart data effects, alert submit and controlled alert form.
frontend/src/styles.css	90-164; 231; 412-507; 510-729; 733-776	Market layout, focus, home responsive/motion, chart and auth responsive rules.

9. A 3-5 minute demonstration plan

Keep the demo focused. Tell the evaluator what interaction you will show, perform it, then point to the responsible source.

Time	Action	Explain / source
0:00-0:30	Open the app and name the stack.	React entry and route shell: main.jsx:1-6; App.jsx:29-57.
0:30-1:10	Open a market overview; change a filter or symbol.	Markets owns quote/history/filter state; row selection updates selectedSymbol: Markets.jsx:10-58.
1:10-2:10	Open /chart/BTCUSDT; change interval/type or indicator; add a drawing.	Workspace props/state and chart resource lifecycle: ChartWorkspace.jsx:142-222, 779-879.
2:10-2:50	Show watchlist or alert form validation and feedback.	Controlled state and async submit path: ChartWorkspace.jsx:582-589 or :722-752.
2:50-3:30	If time, show sign-up/sign-in field validation and explain auth.	AuthPages.jsx:23-56; AuthContext and ProtectedRoute.
3:30-4:15	Show narrow layout or CSS breakpoint, then answer one failure case.	styles.css relevant media query; explain request errors/loading states and cleanup.
4:15-5:00	Summarize your contribution and a next improvement.	Be specific: what you implemented, debugged, and how you verified it.

If a live API or backend is unavailable

Do not conceal the issue or invent output. Name the feature, show the error/loading path or explain the expected flow from source, then state the environmental dependency. A demo can still earn understanding marks if you explain the failure and recovery accurately.

Short opening script

“This is a React and Vite frontend with an Express API. App.jsx owns routes and auth context; page components own screen-specific state. I will show a market filter, the chart workspace cleanup and state flow, and one validated form. I will explain the source locations as I go.”

10. Viva practice - project-specific questions

1. Where does React mount?

Answer: main.jsx creates the root and renders App inside StrictMode. **Follow-up:** What does StrictMode help expose during development?

2. How is the page selected?

Answer: BrowserRouter matches the current URL to the route elements in App.jsx. **Follow-up:** What happens for an unknown URL?

3. Which component is the active chart page?

Answer: ChartWorkspace.jsx is mounted by /chart/:symbol; /chart redirects to BTCUSDT. **Follow-up:** Why should you avoid describing legacy Chart.jsx as the active route?

4. What makes Paper Trading protected?

Answer: The route uses ProtectedRoute and auth context. **Follow-up:** Why must the server also authenticate?

5. What is a controlled input?

Answer: Its value comes from React state and onChange updates that state. Sign-up is an example. **Follow-up:** Name one uncontrolled input in this project.

6. Why does chart symbol input use defaultValue?

Answer: It reads the form DOM value on submit and uses key={symbol} to reset when route symbol changes. **Follow-up:** What validation follows submit?

7. Why use AbortController in Markets?

Answer: Cleanup aborts a stale history request when the selected symbol changes or the page unmounts. **Follow-up:** What race could happen without cleanup?

8. Why is a chart instance held in useRef?

Answer: It is an external mutable object and changing it should not itself render React UI. **Follow-up:** Which other chart objects are kept in refs?

9. What does the chart effect clean up?

Answer: Socket, ResizeObserver, chart subscriptions and the chart instance. **Follow-up:** What bug could missing cleanup cause?

10. How does the app switch the workspace sidebar panel?

Answer: activeTab is state; useMemo returns the corresponding panel with props. **Follow-up:** Why pass onSelectSymbol to WatchlistTab?

11. How are indicators toggled?

Answer: A functional state update flips a single property while preserving the rest. **Follow-up:** Why use the callback form?

12. How does a watchlist symbol get validated?

Answer: Trim, uppercase, remove slash, then match the USDT pair regex; saving requires login. **Follow-up:** What feedback appears on invalid input?

13. Where do client API requests go in development?

Answer: API service uses /api by default and Vite proxies it to the backend. **Follow-up:** Where would a non-default backend origin be configured?

14. How does the API client attach auth?

Answer: It reads the saved token and adds a Bearer Authorization header. **Follow-up:** Why is local validation not security?

15. What is the purpose of AuthContext?

Answer: It makes the current user and auth actions available across routed components. **Follow-up:** Which state should remain local instead?

16. How are loading and empty states shown?

Answer: Shared status components and conditional branches render a clear state instead of a blank panel. **Follow-up:** Give an example with a refresh action.

17. Why do list elements need keys?

Answer: Stable keys help React match items across updates. Market rows use symbol; comment/news lists use identifiers. **Follow-up:** Why is an array index a weaker key when items reorder?

18. How does responsive styling work here?

Answer: CSS media queries reorganize grids and chart/auth layouts at set widths. **Follow-up:** Which file and breakpoint would you inspect?

19. How is reduced motion handled?

Answer: A prefers-reduced-motion rule in styles.css reduces or disables motion. **Follow-up:** Why is that useful?

20. How does a sign-up submit work?

Answer: Prevent default, call register, navigate on success, track busy and error. **Follow-up:** What constraints exist on name/email/password?

21. What does the paper-trading form do?

Answer: State drives the order preview and availability; submit calls the order flow then refreshes portfolio. **Follow-up:** What prevents duplicate order submission?

22. How are drawings persisted?

Answer: Per-symbol drawing state is saved in localStorage. **Follow-up:** What is the limitation versus account persistence?

23. How are community ideas/news loaded?

Answer: Community components request API data, use category/filter state, and render loading/error/empty/list states. **Follow-up:** What does abort cleanup protect against?

24. What does the forex page do when provider data is unavailable?

Answer: It surfaces loading/error/empty feedback; provider credentials and limits are external dependencies. **Follow-up:** Does missing volume get fabricated? No, do not claim provider volume that is absent.

25. What tests ran for this repository?

Answer: Backend Node test suite: 58 passing at preparation time; frontend production build also ran. **Follow-up:** Does that prove third-party live data works? No.

26. What would you improve next?

Answer: Choose one evidenced issue, such as live responsive QA, broader frontend interaction tests, or clearer provider setup. **Follow-up:** How would you verify the change?

11. Debugging, verification, and honest limits

A useful debugging sequence

Step	What to check
1. Reproduce	Write the URL and exact click/input sequence. Check if behavior is route-specific.
2. Inspect UI state	Find the state value, event handler, and conditional render branch.
3. Inspect network/API	Check request path, status, response body, token header, and backend log.
4. Check effect lifecycle	Confirm dependency array, stale request cancellation, timer/listener cleanup.
5. Check environment	Confirm backend port, MongoDB availability, provider key, and Vite proxy.
6. Verify recovery	Repeat action and confirm success, error, empty, or retry state; record what was tested.

Verification recorded while preparing this guide

Backend tests: npm test in backend completed with 58 passing and 0 failing. Frontend production build: npm run build completed successfully with Vite 6.4.5. These checks verify backend test cases and bundle compilation; they do not prove live third-party API availability, database readiness, or responsive visual QA.

Known boundaries to explain accurately

Area	What the code/evidence supports	What not to overclaim
Live data	Provider-backed endpoints are implemented; client renders request states.	That external keys, quotas, network, or markets are always available.
Persistence	Auth uses token/local user state; chart drawings use browser localStorage; some user data uses MongoDB.	That drawings sync across devices or work without backend/database.
Testing	Backend Node tests cover API/domain scenarios; frontend build verifies bundle compilation.	That a frontend test/lint suite ran; frontend package scripts do not include those.
Visual QA	Responsive breakpoints and reduced-motion rules exist in CSS.	That every device/browser width has been manually checked for this guide.
Financial display	Paper Trading is simulation; market data and signals are examples.	That this is brokerage execution or investment advice.

Package scripts and run notes

Frontend package scripts provide dev, build, and preview. Backend package scripts provide dev and test. If backend startup says EADDRINUSE on port 5000, another process is already bound to that port; identify the existing server before starting a duplicate. MongoDB and provider environment variables are separate runtime dependencies.

12. Final preparation checklist

Before the evaluation	Ready
I can explain which route renders each screen.	[ ]
I can show one reusable component and explain its props.	[ ]
I can identify state, event handler, and render branch for one interaction.	[ ]
I can explain one useEffect dependency and its cleanup.	[ ]
I can show a controlled form and explain validation and submit behavior.	[ ]
I can describe how the chart page handles an unavailable request.	[ ]
I can point to one responsive breakpoint and test it live.	[ ]
I can state my individual contribution precisely and honestly.	[ ]
I know the source line references may change if code changes.	[ ]
I can name one limitation and a concrete next step.	[ ]

> Last-minute strategy: choose a reliable feature, rehearse the exact route and clicks, keep the relevant files ready, and explain one implementation decision plus one failure case. Clear reasoning is stronger than racing through every page.

Source map included in this guide: frontend/src/main.jsx; frontend/src/App.jsx; frontend/src/context/AuthContext.jsx; frontend/src/services/api.js; frontend/src/components/Shared.jsx; frontend/src/components/Community.jsx; frontend/src/components/MarketOverview.jsx; frontend/src/pages/Markets.jsx; frontend/src/pages/AuthPages.jsx; frontend/src/pages/Home.jsx; frontend/src/pages/ChartWorkspace.jsx; frontend/src/pages/PaperTrading.jsx; frontend/src/pages/IdeaDetail.jsx; frontend/src/pages/ForexChart.jsx; frontend/src/pages/IndiaMarket.jsx; frontend/src/pages/IndiaChart.jsx; frontend/src/styles.css.